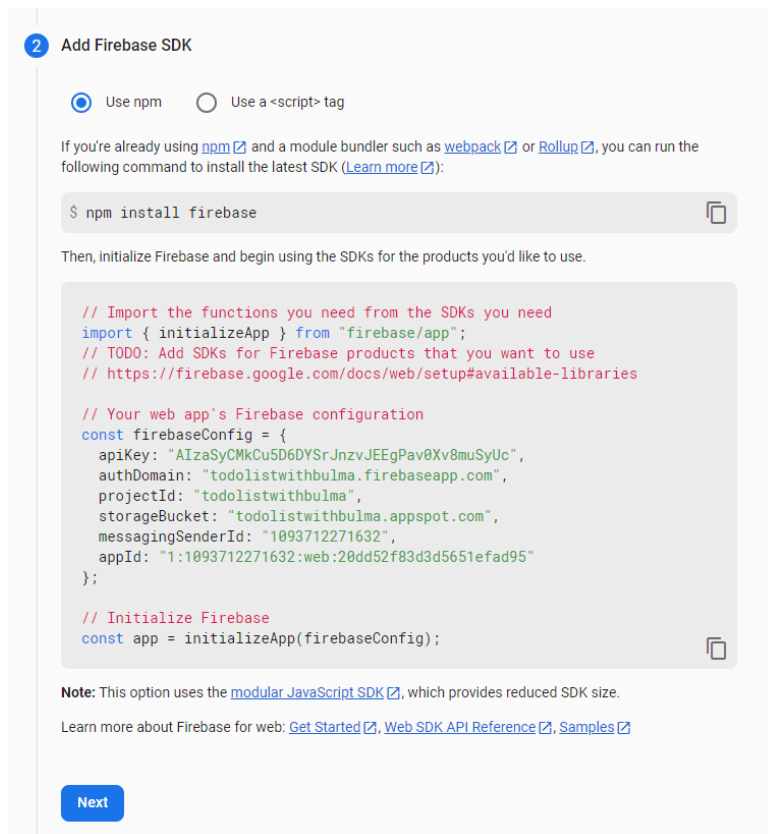
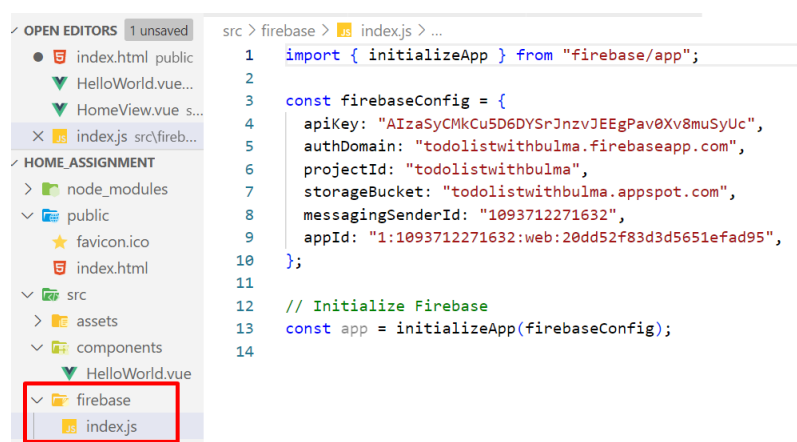


Connecting with Firebase

- 1) <https://firebase.google.com/>
- 2) Log on with your google account
- 3) Go to Console and Add a new project
- 4) Create a web application (App nickname and Register App)



- 5) Run `npm install firebase` in your VSC terminal
- 6) After installing firebase using npm, create a new folder *firebase* under the *src* folder. Then copy the above code and paste in a new file *index.js*. Remove all comments.



FIREBASE: Firestore Setup

- Go To Console – Firestore Database (Under Build menu item)
- Create database – Start in test mode

Create database

1 Secure rules for Cloud Firestore

2 Set Cloud Firestore location

After you define your data structure, you will need to write rules to secure your data.
[Learn more](#)

☐ Start in production mode
Your data is private by default. Client read/write access will only be granted as specified by your security rules.

☒ Start in test mode
Your data is open by default to enable quick setup. However, you must update your security rules within 30 days to enable long-term client read/write access.

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    match /{document=**} {
      allow read, write: if
        request.time < timestamp.date(2023, 6, 11);
    }
  }
}
```

!

The default security rules for test mode allow anyone with your database reference to view, edit and delete all data in your database for the next 30 days

Enabling Cloud Firestore will prevent you from using Cloud Datastore with this project

Cancel **Next**

Cloud Firestore location

eur3 (Europe)

- Create a collection (table) and name it – *todos* (or related to your project)

Start a collection

1 Give the collection an ID

2 Add its first document

Parent path
/

Collection ID [?](#)
todos

Cancel **Next**

- To add a document (record) click on AutoID (Firebase generate an auto id) and add 2 fields

Document parent path
/todos

Document ID [?](#)
633NHDa9HZJ9IH44zdSJ

Field	Type	Value
content	string	Training
done	boolean	true

Go to the Firestore Documentation - <https://firebase.google.com/docs/firestore/quickstart>

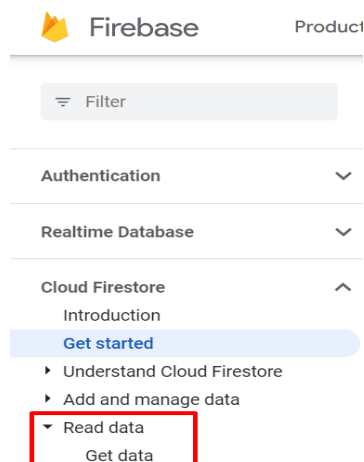
Go to the **Setup your development environment** section and select Web modular API. In your index.js file under the firebase folder in your Application add the following:

```
src > firebase > index.js > ...
1  import { initializeApp } from "firebase/app";
2  import { getFirestore } from "firebase/firestore";
3
4  const firebaseConfig = {
5    apiKey: "AIzaSyCMkCu5D6DYSrJnzvJEEgPav0Xv8muSyUc",
6    authDomain: "todolistwithbulma.firebaseio.com",
7    projectId: "todolistwithbulma",
8    storageBucket: "todolistwithbulma.appspot.com",
9    messagingSenderId: "1093712271632",
10   appId: "1:1093712271632:web:20dd52f83d3d5651efad95",
11 };
12
13 // Initialize Firebase
14 const app = initializeApp(firebaseConfig);
15
16 // Initialize Cloud Firestore and get a reference to the service
17 const db = getFirestore(app);
18 export {
19   db
20 }
```

Displaying todos

In HomeView.vue add – `import { db } '@/firebase'. @is an alias to the src folder`

Go to the Firebase documentation – Read data – Get data/Get real-time updates (<https://firebase.google.com/docs/firestore/>)



With the below you will get the data and show it in the console.

```
import {
  getDocs,
  collection,
} from "firebase/firestore";
```

```

onMounted(async() => {
  //this will fire the first time the app is launched
  const querySnapshot = await getDocs(collection(db, 'todos'))
  querySnapshot.forEach((doc) => {
    console.log(doc.id, " => ", doc.data());
  })
})

```

```

jYtZCTho6QQGpNqv67T2 => ▶ {done: false, content: 'Studying'}
sDWlw2ZqKPyBzzGjElgn => ▶ {done: true, content: 'Training'}

```

Add the data retrieved to the todos array.

In the Firestore documentation under [Listen to multiple documents in a collection](#) check onSnapshot.

Use **onSnapshot** (Get real-time updates) instead of getDocs so whenever there is a change it will keep on listening and fire again immediately.

Adding todos

Inside the Firestore documentation – Add data – Add a document and change the function todo.

```

function addToDo() {
  // Add a new document in collection "todos"
  addDoc(collection(db, "todos"), {
    content: inputField.value,
    done: false
  });
  inputField.value = ""
};

```

Refer to examples on VLE

Firestore Hosting

Install Firebase CLI to deploy Firestore Hosting - `npm install -g firebase-tools`. Make sure you are running Visual Studio Code as an administrator. If you do `firebase -V` you should be able to view the version which was installed.

If you have issues with the installation take a look at the [Firestore CLI Reference](#) or change your [npm permissions](#).

Deploy to Firestore Hosting. Inside the root folder of your project do the following:

You can deploy now or [later](#). To deploy now, open a terminal window, then navigate to or create a root directory for your web app.
Sign in to Google

```
$ firebase login
```

Initiate your project
Run this command from your app's root directory:

```
$ firebase init
```

When you're ready, deploy your web app
Put your static files (e.g., HTML, CSS, JS) in your app's deploy directory (the default is "public").
Then, run this command from your app's root directory:

```
$ firebase deploy
```

After deploying, view your app at [todolistwithbulma.web.app](#)
Need help? Check out the [Hosting docs](#)

When doing `firebase init` you will be asked a number of questions. When asked whether you want to use `public` as your public directory type in **dist**

```
select, <a> to toggle all, <i> to invert selection, and <enter> to proceed)
( ) Realtime Database: Configure a security rules file for Realtime Database and (optionally) provision default
( ) Firestore: Configure security rules and indexes files for Firestore
( ) Functions: Configure a Cloud Functions directory and its files
>(*) Hosting: Configure files for Firestore Hosting and (optionally) set up GitHub Action deploys
( ) Hosting: Set up GitHub Action deploys
( ) Storage: Configure a security rules file for Cloud Storage
( ) Emulators: Set up local emulators for Firebase products
(Move up and down to reveal more choices)

? What do you want to use as your public directory? dist
? Configure as a single-page app (rewrite all urls to /index.html)? Yes
? Set up automatic builds and deploys with GitHub? No
+ Wrote dist/index.html

i Writing configuration info to firebase.json...
i Writing project information to .firebaserc...
```

Refer to this video tutorial: <https://www.youtube.com/watch?v=a-0W40NM0dI&t=151s>

Then do `npm run build` to generate the `dist` folder in your project.

- Finally in the terminal write: **`firebase deploy`**
- If you go to Hosting you will have 2 domains

Hosting

[Dashboard](#) [Usage](#)

todolistwithbulma domains

Domain	Status
todolistwithbulma.web.app Default	
todolistwithbulma.firebaseio.com Default	